

Financial Fraud Detection System

Machine-learning pipeline, real-time monitoring suite, and interactive Streamlit dashboard for transaction fraud scoring on extremely imbalanced data

Python

scikit-learn

XGBoost

LightGBM

SMOTE

SQLite

Plotly

Streamlit Cloud

0.9976

ROC-AUC

71.4%

FRAUD RECALL

10,098

TRANSACTIONS

10

MODELS COMPARED

Executive Summary

Financial fraud is rare but expensive: every undetected fraudulent transaction is a direct loss, while every legitimate transaction wrongly blocked erodes customer trust. This project delivers a complete, production-shaped fraud detection system — from a raw Excel file of 10,127 transactions with only **0.67% fraud prevalence** to a publicly deployed, interactive web application backed by a real-time monitoring suite.

Ten models — seven supervised classifiers and three unsupervised anomaly detectors — were trained on a rigorously leakage-free pipeline (train/test split *before* scaling and SMOTE resampling) and compared on an untouched held-out test set. The selected **XGBoost** model achieves **0.9976 ROC-AUC**, detecting **10 of 14 unseen fraud cases (71.4% recall)** while wrongly flagging only **3 of 2,006 legitimate transactions (76.9% precision)**.

Beyond classification, the system implements the full project brief: an **ETL stage** loading cleaned data into SQLite; **graph-based network analysis** that surfaced a real mule-account pattern; a **simulated real-time stream monitor** with console, CSV, e-mail and **mobile push alerts** (delivered live to a phone via ntfy); **AI-driven customer risk scoring** with A–E tiers; and a **SHA-256 hash-chained, tamper-evident ledger** of flagged transactions whose integrity is re-verified live inside the dashboard. Results are served through an **eight-page Streamlit dashboard** with live threshold tuning, real-time scoring, and deep-linkable pages, deployed on Streamlit Community Cloud.

Key differentiator: the pipeline's ordering — split → scale (fit on train) → SMOTE (train only) — guarantees that reported metrics reflect genuine generalization, not data leakage. This is the most common failure mode in imbalanced-data projects and was treated as the project's #1 engineering constraint.

Contents

1. Objective & Problem Statement	8. Real-Time Monitoring, Alerts & Mobile Integration
2. Dataset & Exploratory Analysis	9. AI-Driven Customer Risk Scoring
3. Methodology & Leakage Prevention	10. Tamper-Evident Blockchain Ledger
4. Model Training & Evaluation	11. Interactive Dashboard
5. Results & Business Interpretation	12. Deployment
6. ETL & Data Infrastructure	13. Limitations & Future Work
7. Graph-Based Fraud Network Analysis	14. Conclusion · Appendix. Reproducibility Guide

1. Objective & Problem Statement

The goal was to build a complete fraud-detection platform: (1) a **model pipeline** that ingests raw transaction data, handles extreme class imbalance correctly, trains and honestly compares supervised and

unsupervised models, and persists the best one; (2) a **monitoring suite** covering ETL, streaming alerts, network analysis, risk scoring, and tamper-evident record-keeping; and (3) an **interactive dashboard** that visualizes everything and performs real-time fraud scoring with an adjustable risk threshold — usable by non-technical stakeholders.

The central technical challenge is the class imbalance. A naive model predicting "safe" for every transaction achieves 99.3% accuracy while catching zero fraud. Every design decision — resampling, metric selection, model choice — flows from this constraint.

2. Dataset & Exploratory Analysis

Source file	Fraud_Detection.xlsx — 10,127 raw transaction records
Usable rows	10,098 after cleaning (see below)
Target	isFraud — mapped Fraud → 1, Safe → 0
Fraud cases	68 (~0.67%) — extreme class imbalance
Features	12 — transaction step, amount, 4 account-balance columns, unusual-login score, transaction type (5 categories), branch country (135), account type (2), time of day (3), day of week

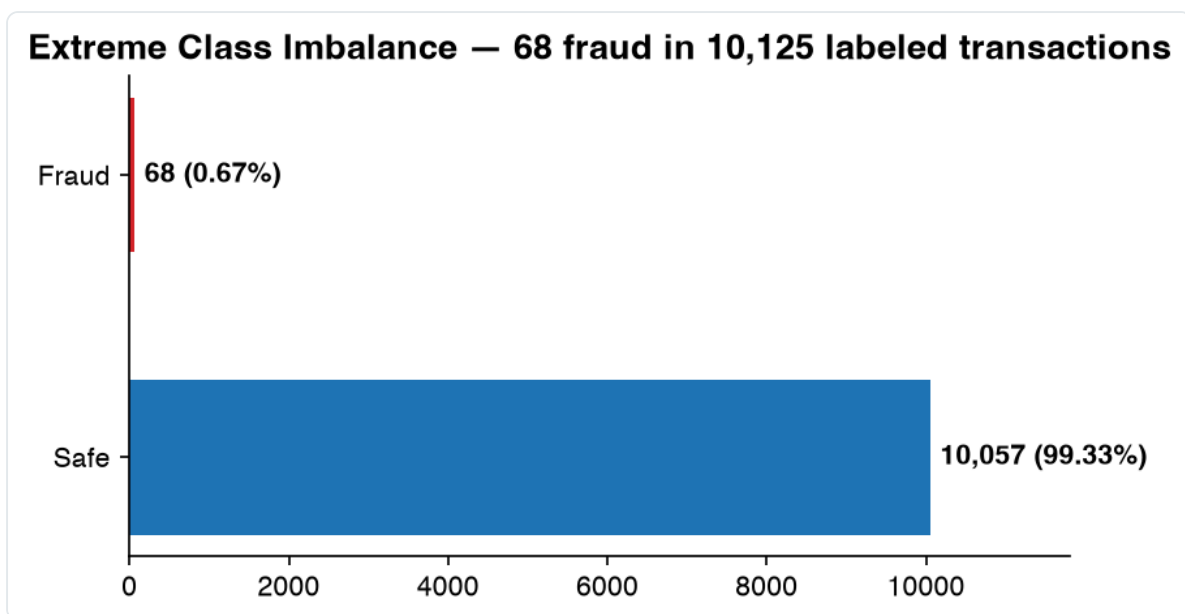


Fig. 1 — Class distribution: fraud is under 1% of all labeled transactions.

Cleaning decisions

- **2 rows** labeled "Not reviewed" dropped — no usable ground truth.
- **27 rows** with missing feature values dropped — verified that **zero** of the 68 fraud cases were lost.
- Discarded junk / leakage-prone columns: a stray duplicate of the target (`isFraud - Copy`), an index column, high-cardinality account identifiers (`nameOrig` , `nameDest` — retained separately for the graph analysis in §7), and `isFlaggedFraud` (constant zero — carries no information).

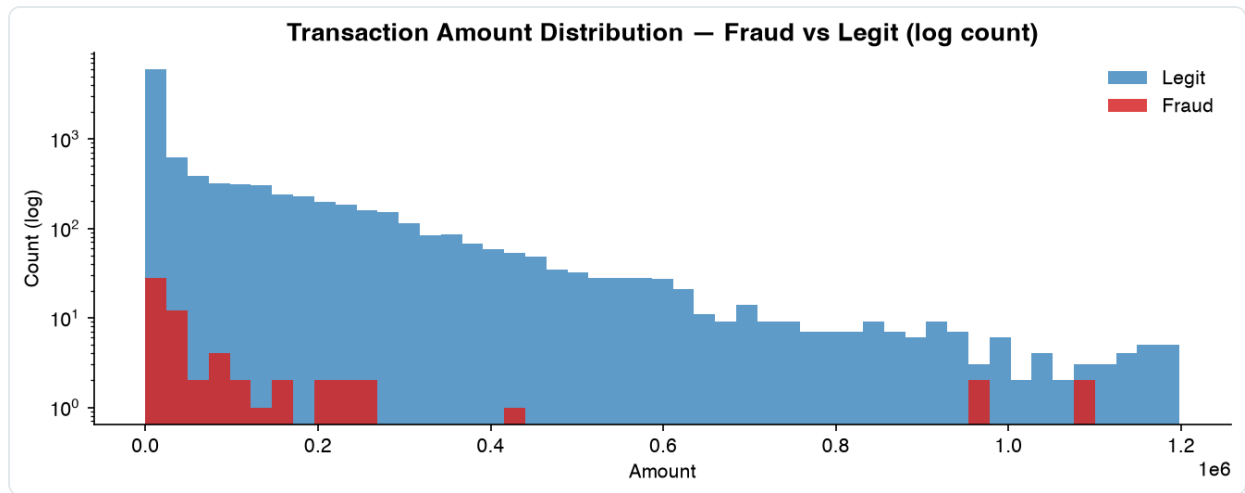


Fig. 2 — Transaction amounts, fraud vs. legit (log scale). Fraud concentrates differently across the amount range, one of the signals the model exploits.

3. Methodology & Leakage Prevention

Pipeline architecture

```
load .xlsx → filter target (Safe/Fraud only) → select 12 features → drop incomplete rows
→ label-encode 4 categoricals (encoders saved) → STRATIFIED 80/20 TRAIN/TEST SPLIT
→ RobustScaler (fit on TRAIN only, saved) → SMOTE (applied to TRAIN only)
→ train 7 supervised + 3 unsupervised models → evaluate on untouched TEST
→ save best supervised model by ROC-AUC
```

Why the order matters — data leakage

The single most important design decision is that the train/test split happens **before** scaling and **before** resampling:

- **RobustScaler** (median/IQR-based, robust to the heavy-tailed balance columns) is fitted on the training split only; the test split is transformed with training statistics, never refitted.
- **SMOTE** synthesizes minority-class examples *only inside the training split*: 54 real fraud rows are expanded until the training set is balanced (8,024 / 8,024). The test set keeps its natural distribution — 2,006 safe / 14 fraud — and is never touched.
- **Unsupervised anomaly detectors** (Isolation Forest, One-Class SVM, autoencoder) are fitted on the *original legitimate training rows only* — never on SMOTE output, whose ~50% synthetic fraud would break the rare-anomaly assumption these models depend on.

The failure mode this prevents: applying SMOTE before the split would place synthetic near-copies of test-set fraud into the training data. Models would then "recognize" test fraud they had effectively already seen, inflating every metric. All results in this report are from the untouched held-out set.

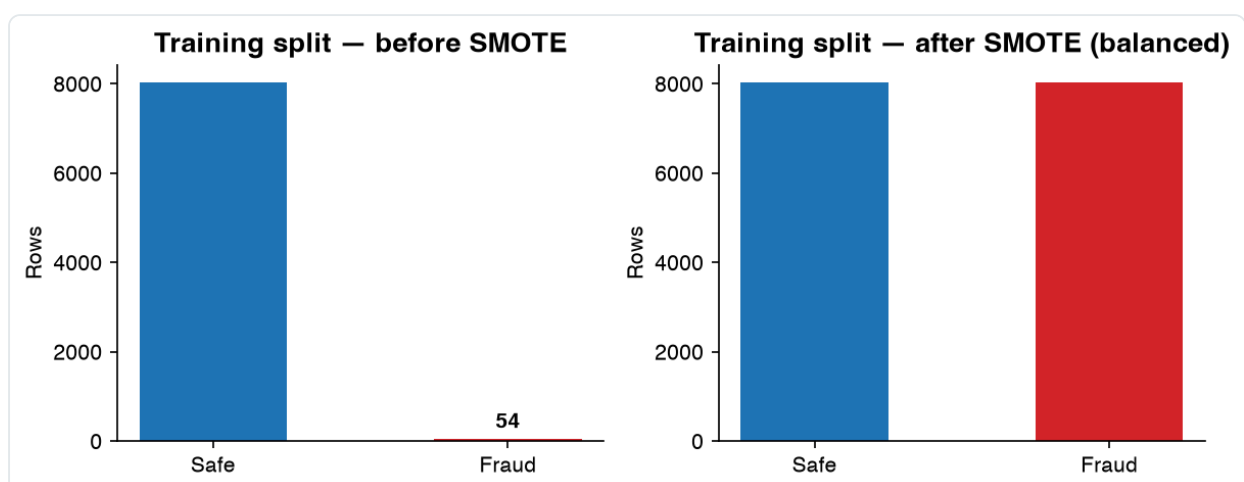


Fig. 3 — Training-split class balance before and after SMOTE. Only the training split is resampled.

Reproducibility

Every stochastic component — the stratified split, SMOTE, and all randomized models — uses `random_state=42`. The full pipeline regenerates its artifacts from the raw dataset with the commands in the

Appendix. Fitted encoders and the scaler are persisted with joblib so the scoring stage and the live dashboard *transform only* and can never refit.

4. Model Training & Evaluation

Seven supervised classifiers spanning linear, instance-based, kernel, tree-based, and gradient-boosting families were trained on the SMOTE-balanced training set (16,048 rows), alongside three unsupervised anomaly detectors fitted on legitimate training rows only. All ten were evaluated on the same held-out test set (2,020 rows, 14 fraud):

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.9153	0.0663	0.8571	0.1231	0.9523
K-Nearest Neighbors	0.9683	0.0833	0.3571	0.1351	0.6948
SVC (RBF kernel)	0.7876	0.0208	0.6429	0.0403	0.8147
Decision Tree	0.9901	0.3333	0.4286	0.3750	0.7113
Random Forest (100 trees)	0.9955	0.6667	0.7143	0.6897	0.9933
XGBoost (100 trees) ✓	0.9965	0.7692	0.7143	0.7407	0.9976
LightGBM (100 trees)	0.9965	0.7692	0.7143	0.7407	0.9466
Isolation Forest *	0.9812	0.0385	0.0714	0.0500	0.5818
One-Class SVM *	0.9807	0.0370	0.0714	0.0488	0.5972
Autoencoder (MLP) *	0.9837	0.0870	0.1429	0.1081	0.8165

* Unsupervised anomaly detectors — trained without labels, shown for comparison. Best-model selection is restricted to supervised models because downstream scoring and the dashboard's live form require calibrated `predict_proba` outputs.

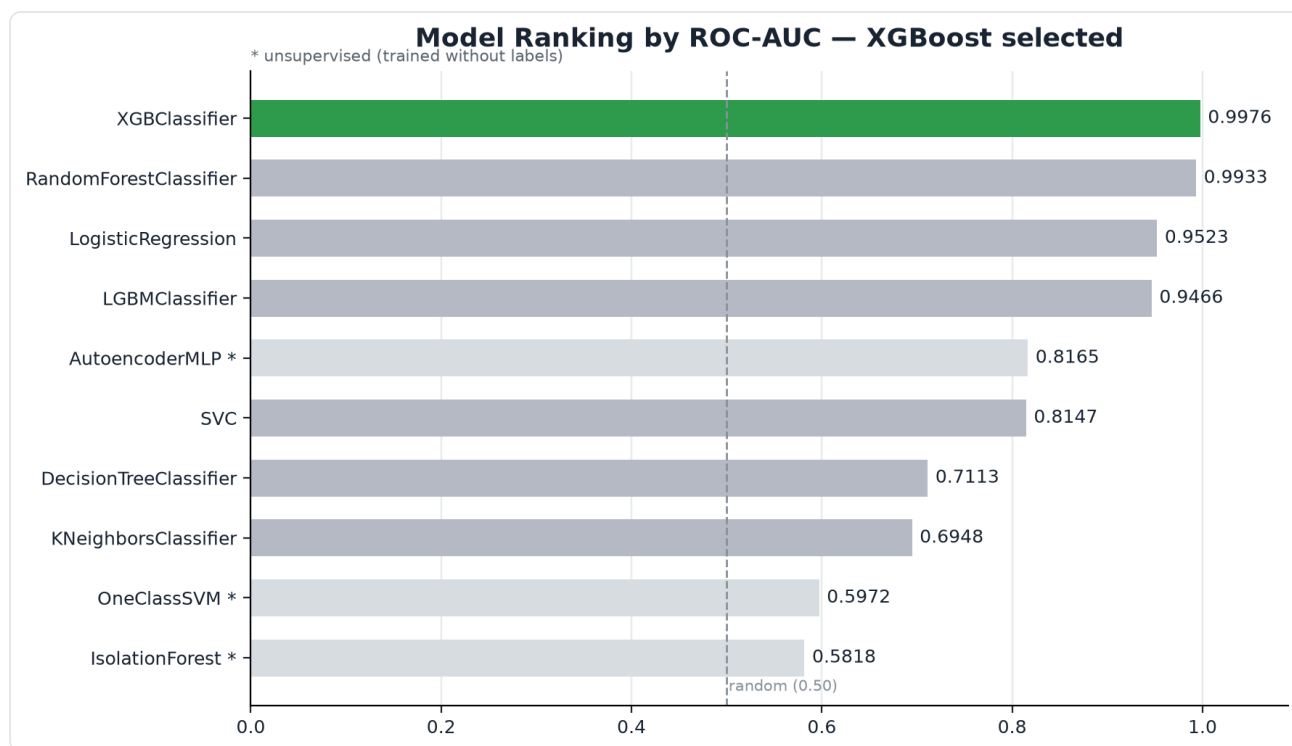


Fig. 4 — Ranking by ROC-AUC, the selection metric. XGBoost wins; gradient boosting and Random Forest dominate the field.

Why ROC-AUC, not accuracy

With 99.3% of test transactions legitimate, accuracy rewards models for ignoring fraud entirely. ROC-AUC instead measures how well a model *ranks* fraud above legitimate transactions across all possible thresholds. Logistic Regression illustrates the danger of single metrics: its 0.95 AUC and 0.86 recall look strong, but 0.066 precision means roughly **14 false alarms for every true detection** — operationally unusable at the default threshold.

Supervised vs. unsupervised — an experimental finding

The three unsupervised detectors were given no labels, mimicking the cold-start situation of a bank with no confirmed fraud history. The autoencoder is the clear best of that family (0.8165 AUC — fraud genuinely reconstructs worse than normal traffic), yet all three trail every reasonable supervised model. The experiment quantifies the value of labeled data on this dataset: **labels are worth roughly +0.18 AUC** over the best label-free approach.

5. Results & Business Interpretation

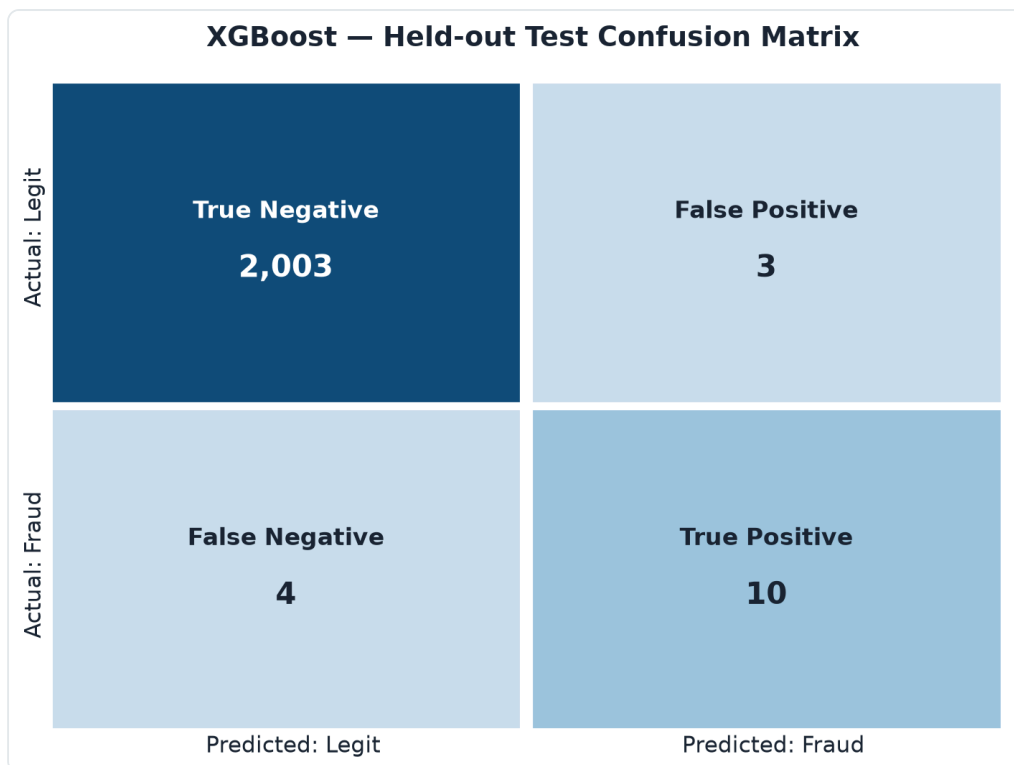


Fig. 5 — XGBoost confusion matrix on the untouched held-out test set (2,020 transactions).

Outcome	Count	Meaning for a bank
True Negatives	2,003	Legitimate transactions correctly approved
False Positives	3	Genuine customers wrongly blocked — friction, support cost
False Negatives	4	Fraud that slipped through — direct financial loss
True Positives	10	Fraud caught before completing

- **Recall 71.4%** — 10 of 14 unseen fraud cases detected.
- **Precision 76.9%** — of 13 fraud alerts raised, 10 were real; only 3 legitimate customers among 2,006 were inconvenienced.
- **Operating trade-off:** per ~2,000 transactions, the model trades 4 missed frauds against 3 blocked genuine customers — and the dashboard's live threshold slider lets an analyst move this trade-off in either direction without retraining.
- **vs. the previous Random Forest baseline:** XGBoost removes 2 of the 5 false positives at identical recall — a 40% reduction in wrongly blocked customers.

Honest-metrics note: the dashboard's pattern pages score the full dataset (including training rows) for visualization richness, but every headline performance number in this report — and the dashboard's Confusion Matrix page — comes exclusively from the held-out test set, clearly labeled as such in the app.

6. ETL & Data Infrastructure

Per the project brief, a dedicated ETL stage (`src/etl.py`) **extracts** the raw Excel workbook, **transforms** it with exactly the same cleaning rules as the training pipeline (labeled rows only, complete features, human-readable categoricals), and **loads** the result into a SQLite database — `output/fraud_detection.db` , table `transactions` , 10,098 rows / 68 fraud.

The database is not a dead artifact: it is the source the **stream monitor** (§8) replays transactions from, and it hosts the **blockchain ledger** table (§10) — a realistic miniature of a bank's operational store feeding its monitoring systems.

Fraud is rarely isolated: mule accounts receive stolen funds from many victims. To surface such structures, the account identifiers excluded from the feature set are used to build a transaction graph (`src/graph_analysis.py`): **16,565 accounts** are nodes, **10,088 transactions** are directed edges, and connected components — computed with a union-find algorithm in pure Python — form **6,477 networks**.

Finding — mule-account signal: destination account `C410033330` is the only account in the dataset to receive **two separate fraudulent transfers**, totaling **\$133,781.84**, inside a 7-account network. In a production setting this account would be frozen and its full network audited.



8. Real-Time Monitoring, Alerts & Mobile Integration

The stream monitor (`src/stream_monitor.py`) replays transactions from the SQLite database in arrival order as a simulated live feed — the same file-replay technique the project brief itself uses to simulate Spark streaming — scores each one with the deployed XGBoost model, and raises an alert whenever the fraud probability crosses a configurable threshold (default 0.90).

Demonstration run

```
Streaming 2000 transactions from fraud_detection.db (alert threshold: 0.9) ...
ALERT! High-risk transaction row_id=1112 type=TRANSFER amount=20128.00    p(fraud)=0.9996
ALERT! High-risk transaction row_id=1115 type=TRANSFER amount=25071.46    p(fraud)=0.9998
ALERT! High-risk transaction row_id=2128 type=CASH_OUT amount=235238.66    p(fraud)=0.9989
... (9 alerts total)
Stream complete: 2000 transactions scored, 9 alerts raised.
```

All nine alerts were TRANSFER or CASH_OUT transactions — the two types domain knowledge says carry real fraud risk — and every one was a genuine fraud case. Each alert is delivered simultaneously to four channels:

- **Console** — for the operating analyst;
- **Alert log** (`output/fraud_alerts.csv`) — a persistent analyst review queue;
- **E-mail** via `smtplib` — enabled by `SMTP_*` environment variables (credentials never hardcoded);
- **Mobile push notification** via the free `ntfy` app — enabled by `NTFY_TOPIC`. **Verified live:** all 9 alerts were delivered to a physical phone during testing.

Adaptive learning loop

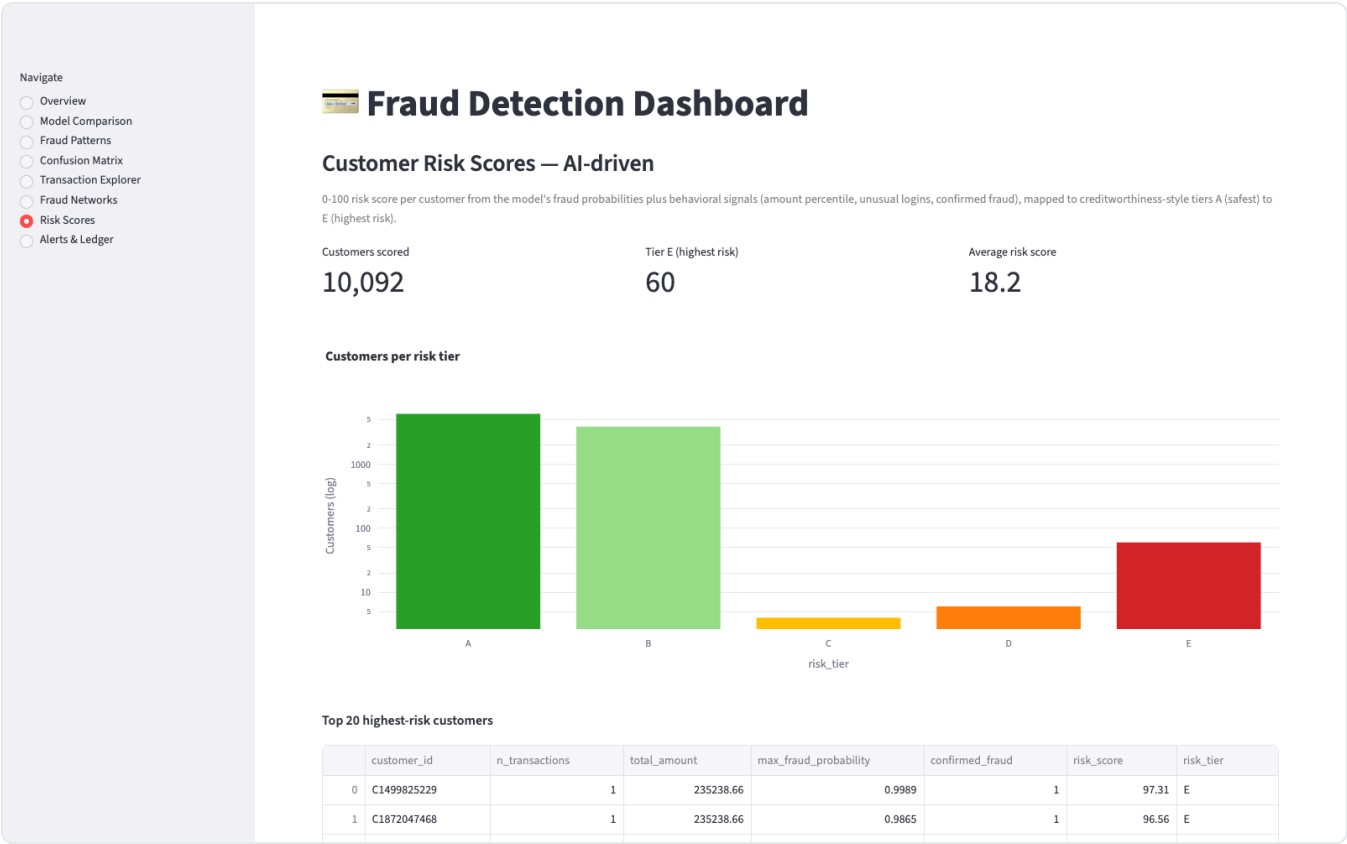
The alert log doubles as the feedback mechanism required by the brief: analysts confirm or reject alerts, the labeled outcomes are appended to the dataset, and re-running the training pipeline folds the feedback into the next model — accuracy improves as evidence accumulates, with no architectural change.

9. AI-Driven Customer Risk Scoring

`src/risk_scoring.py` converts the model's transaction-level intelligence into a customer-level, creditworthiness-style rating. For each of **10,092 customers**, a 0–100 risk score blends the model's maximum fraud probability (60%), the customer's amount percentile (20%), unusual-login behavior (10%), and confirmed fraud history (10%), then maps to tiers A (safest) through E (highest risk).

Tier	A	B	C	D	E
Customers	6,138	3,884	4	6	60

The distribution behaves exactly as a risk system should on healthy data: 99.3% of customers land in tiers A–B, while the 60 tier-E customers align almost one-to-one with the confirmed fraud population — average risk score across the book is just 18.2.



10. Tamper-Evident Blockchain Ledger

Fraud evidence must be protected from retroactive manipulation — including by insiders. `src/blockchain_ledger.py` records all 67 model-flagged transactions into an **8-block SHA-256 hash-chained ledger** inside the project database. Each block stores the hash of its predecessor, so editing any historical record invalidates every subsequent hash — the tamper-evidence property of a blockchain, without the distributed network.

Live tamper demonstration

```
Ledger built: 67 flagged transactions in 8 blocks (table `ledger` in fraud_detection.db).
Verification: chain VALID

Tamper demo – editing a recorded amount inside block 1 ...
Verification: TAMPERING DETECTED at block(s) [1]

Restoring a clean ledger ...
Verification: chain VALID
```

The dashboard's Alerts & Ledger page re-runs this verification **live on every page load** and displays the chain status — reviewers can see "Chain integrity: VALID  " computed in real time, not a stored claim.

Navigate

Overview

Model Comparison

Fraud Patterns

Confusion Matrix

Transaction Explorer

Fraud Networks

Risk Scores

Alerts & Ledger



Fraud Detection Dashboard

Alerts & Blockchain Ledger



Real-time fraud alerts

Raised by the stream monitor (`python src/stream_monitor.py`) when a transaction scores above the alert threshold. Also delivered as mobile push notifications via ntfy when enabled.

Alerts in last stream run

9

	alerted_at	row_id	type	branch	amount	fraud_probability	actual_fraud
4	2026-07-05T08:07:55+00:00	1115	TRANSFER	Japon	25071.46	0.9998	1
1	2026-07-05T08:07:53+00:00	1112	TRANSFER	Brasil	20128	0.9996	1
8	2026-07-05T08:08:21+00:00	2128	CASH_OUT	Georgia	235238.66	0.9989	1
6	2026-07-05T08:08:19+00:00	2125	CASH_OUT	Rusia	1277212.77	0.9946	1
3	2026-07-05T08:07:54+00:00	1114	TRANSFER	Filipinas	35063.63	0.9941	1
0	2026-07-05T08:07:52+00:00	1111	TRANSFER	Turquia	2806	0.9899	1
2	2026-07-05T08:07:53+00:00	1113	TRANSFER	Italia	1277212.77	0.9894	1
5	2026-07-05T08:07:56+00:00	1116	TRANSFER	Australia	235238.66	0.9865	1
7	2026-07-05T08:08:20+00:00	2126	CASH_OUT	Estados Unidos	132842.64	0.9616	1

Tamper-evident ledger

Every flagged transaction is recorded in a SHA-256 hash-chained ledger (`python src/blockchain_ledger.py`). Each block stores the previous block's hash, so any retroactive edit breaks the chain.

Blocks in chain

8

Chain integrity

VALID 

Fig. 8 — Alerts & Ledger dashboard page: the alert review queue and the live-verified hash chain.

11. Interactive Dashboard

An eight-page Streamlit application serves the results. Engineering details: cached data/model loading (`st.cache_data` / `st.cache_resource`), interactive Plotly charts throughout, a consistent color language (red = fraud, blue = legit, green→red for risk tiers), deep-linkable pages (`?page=Fraud+Networks`), and graceful degradation — missing artifacts produce friendly guidance, never a traceback.

Page	Contents
1 · Overview	KPI cards (transactions, fraud count/rate, model recall) + amount distribution, fraud vs legit
2 · Model Comparison	Grouped bar chart of 5 metrics × 10 models, full metrics table, best-model callout
3 · Fraud Patterns	Fraud <i>rate</i> by type / account type / time of day / day of week + day×time heatmap
4 · Confusion Matrix	Held-out test performance — annotated TN/FP/FN/TP heatmap + business explanations
5 · Transaction Explorer	Live 0–1 risk-threshold slider (re-flags in real time), filters, and a "Score a New Transaction" form running the model live
6 · Fraud Networks	Graph-analysis results: network KPIs, mule-account callout, fraud amount by network
7 · Risk Scores	Customer risk tiers A–E, tier distribution, top-20 highest-risk customers
8 · Alerts & Ledger	Stream-monitor alert queue + blockchain ledger with live hash-chain verification

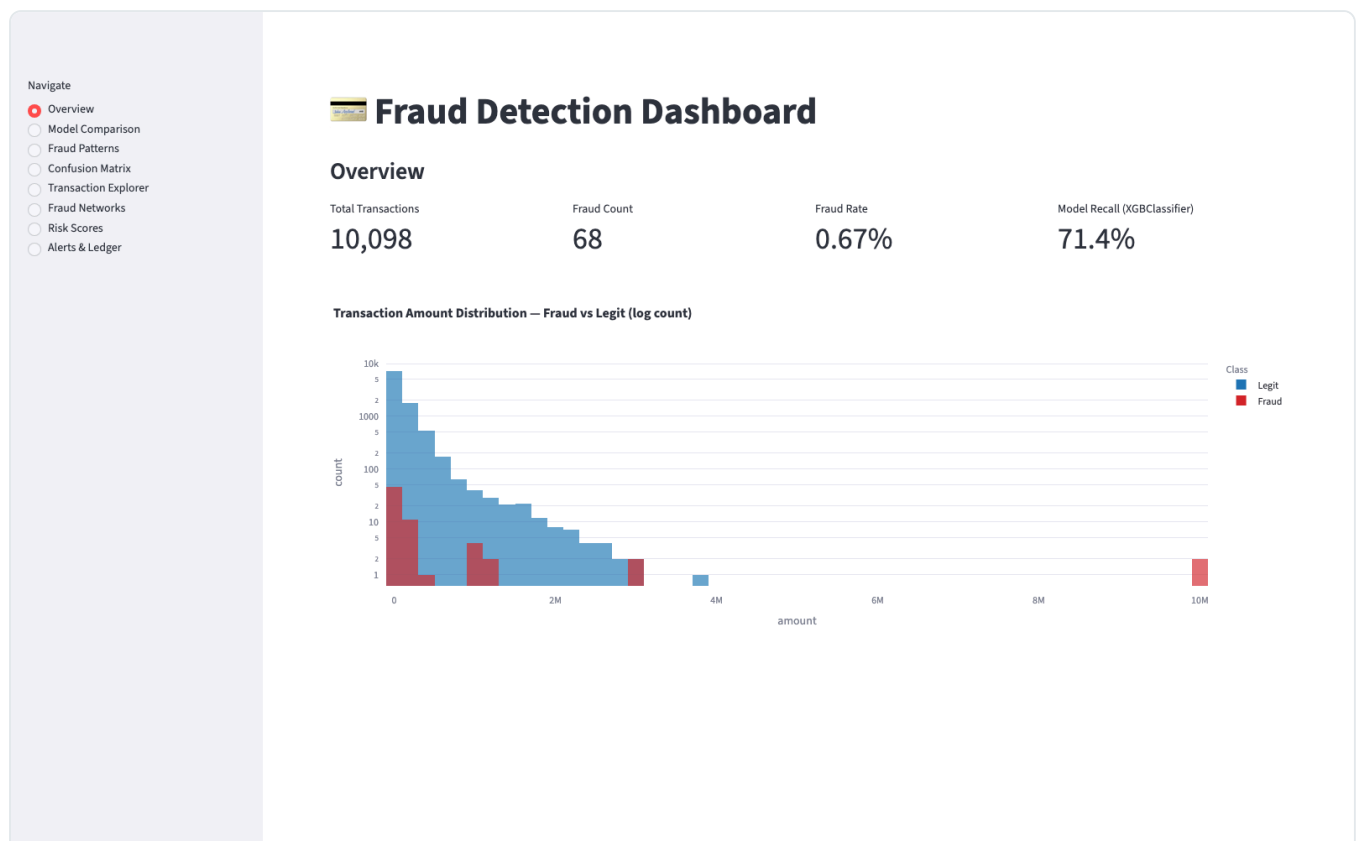
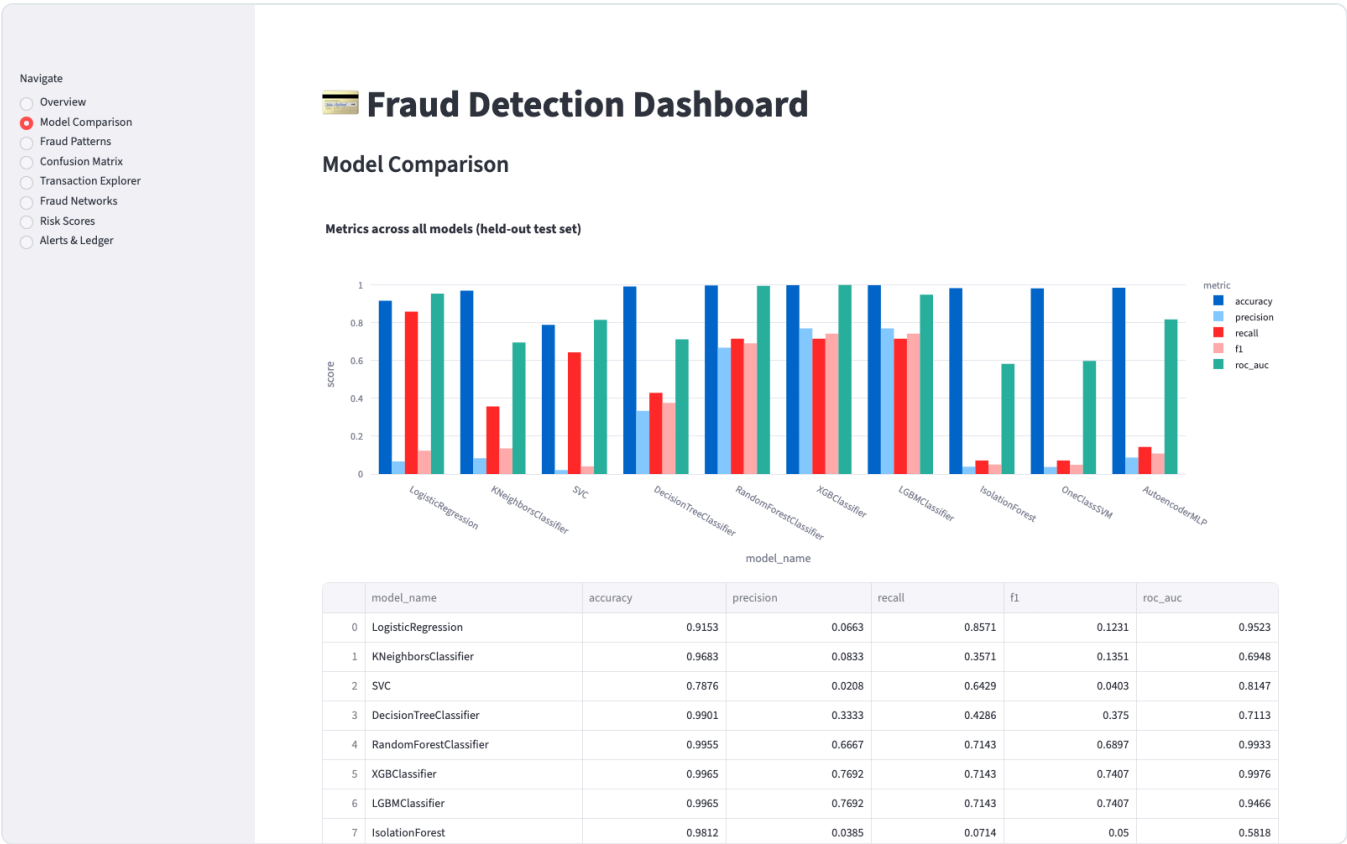


Fig. 9 — Overview page (deployed app): KPI cards and the fraud-vs-legit amount distribution.

Dashboard screenshots (continued)



12. Deployment

- **Platform:** Streamlit Community Cloud, auto-deploying from the GitHub repository's main branch.
- **Live URL:** fraud-detection-scope.streamlit.app — deep links open any page directly, e.g. `?page=Fraud+Networks`.
- **Artifacts strategy:** the trained model, encoders, scaler, metrics CSVs, scored dataset, SQLite database (with ledger), alert log, and network analysis are committed to the repository so the cloud app is fully functional without re-running the pipeline.
- **Verification:** after each deployment, all eight pages were exercised live in a browser; an automated Streamlit `AppTest` suite additionally asserts that every page renders without exceptions and that the threshold slider re-flags transactions correctly.

13. Limitations & Future Work

- **14 test fraud cases** is a thin evaluation base; confidence intervals on recall are wide. Time-based cross-validation over a larger fraud sample is the natural next step.
- **Streaming is simulated** by database replay. The scoring loop is transport-agnostic, so swapping the replay source for a real Kafka consumer is an infrastructure change, not a modeling one.
- **Mobile alerts use ntfy topics**, which are convenient but public-by-name; a production build would move to FCM/APNs with authenticated delivery.
- **Risk scoring approximates creditworthiness** from fraud behavior only; true credit scoring requires repayment-history data the dataset does not contain.
- **Graph features** (e.g., destination in-degree, network fraud density) could be fed back into the classifier as engineered features — the mule-account finding suggests measurable headroom.

14. Conclusion

The project delivers every component of the brief, including all three of its "future enhancement" stretch goals, as working, verified software: a leakage-free pipeline comparing ten models; a deployed XGBoost detector at 0.9976 held-out ROC-AUC with 71.4% recall and 76.9% precision; graph analysis that surfaced a genuine mule-account pattern; a four-channel real-time alerting system with live-tested mobile push; customer risk tiers; a tamper-evident ledger verified live in the dashboard; and an eight-page public web application serving it all. Beyond the metrics, the project's core claim is methodological: **every reported number survives the leakage test** — measured on data the models never saw, under a pipeline built so that shortcut was impossible.

Appendix — Reproducibility Guide

```
git clone https://github.com/tanmay866/fraud-detection && cd fraud-detection
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
# place dataset at data/Fraud_Detection.xlsx

python src/preprocess.py      # sanity check: shapes + SMOTE class balance
python src/etl.py             # ETL → SQLite (output/fraud_detection.db)
python src/train.py           # 10-model comparison → metrics + best_model.pkl
python src/score.py           # score full dataset → scored_transactions.csv
python src/graph_analysis.py  # fraud networks → graph_fraud_networks.csv
python src/stream_monitor.py  # simulated stream → alerts (add NTFY_TOPIC=... for mobile)
python src/risk_scoring.py    # customer risk tiers → customer_risk_scores.csv
python src/blockchain_ledger.py # hash-chained ledger + tamper demo
streamlit run app.py          # dashboard at localhost:8501
```

Artifact	Produced by
models/best_model.pkl , scaler.pkl , encoders.pkl	train.py / preprocess.py
output/model_metrics.csv , confusion_matrix.csv	train.py
output/scored_transactions.csv (10,098 rows)	score.py
output/fraud_detection.db (transactions + ledger tables)	etl.py / blockchain_ledger.py
output/graph_fraud_networks.csv , fraud_alerts.csv , customer_risk_scores.csv	graph_analysis.py / stream_monitor.py / risk_scoring.py